SIMATS ENGINEERING
ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: COMPUTER VISION COURSE CODE: ITA0515 Slot B

COURSE OUTCOME COVERED
CO5: ***Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos.*** **(BTL 6)**

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks 50 Duration :60 Minutes Annexure A
Constraint-Based Problem Solving

Code of Conduct:

I PAVAN JANA (192472313) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Pavan j

Rubrics for Evaluation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	*Marks(100)*
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly	
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization	
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints	

Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts	
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided	

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

1. Real-Time Face Recognition System (Edge Deployment)
A company wants to deploy a face recognition system on low-power edge devices (e.g., Raspberry Pi). The system must:
A. Process live video streams in real time (<30 FPS latency)
B. Work under varying lighting conditions
C. Use limited memory and CPU resources
D. Detect and recognize faces from a predefined dataset
Design a complete solution using OpenCV, specifying: image acquisition, preprocessing, feature detection, and recognition pipeline. Justify how your design satisfies all constraints.

2. Automated Video Surveillance with Alert System
A security firm needs a video surveillance system that:
A. Detects motion and unusual activities
B. Generates alerts in real time
C. Works with standard CCTV video feeds
D. Minimizes false positives in dynamic environments
Design an OpenCV-based system integrating video processing, feature detection, and motion analysis. Justify how your approach handles noise, dynamic backgrounds, and real-time constraints.

3. OCR System for Low-Quality Documents
A digitization company needs an OCR system that:
A. Extracts text from low-resolution, noisy scanned images
B. Handles varying fonts and illumination
C. Works efficiently for batch processing
Design an image processing pipeline using OpenCV functions (enhancement, segmentation, etc.) for OCR. Justify how each stage improves recognition under given constraints.

4. Facial Expression Recognition for Mobile Application
A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints: A. Limited processing power (mobile CPU)
B. Real-time performance
C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

5. Smart Traffic Monitoring System

A city authority needs a system to:

A. Detect vehicles from live video streams

B. Track movement across frames

C. Count vehicles and display statistics

D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

Constraint-Based Problem Solving Using OpenCV

Introduction

A computer vision pipeline is a sequence of processing stages that converts raw visual data into useful information. A typical OpenCV-based pipeline can be represented as:

Input Acquisition → Preprocessing → Detection → Feature Extraction → Recognition/Analysis → Decision → Output

When designing a computer vision system, the algorithm should not only produce accurate results but should also satisfy practical constraints such as:

- Processing speed
- Memory consumption
- CPU usage
- Lighting variations
- Noise
- Accuracy
- False-positive rate
- Continuous operation

OpenCV provides efficient functions for image acquisition, filtering, color conversion, feature detection, object detection, tracking, and visualization.

1. Real-Time Face Recognition System for Edge Deployment

1.1 Problem Analysis

The company wants to deploy a face recognition system on a low-power device such as a Raspberry Pi.

The system receives a continuous video stream from a camera. It must identify whether the detected face belongs to a person in a predefined dataset.

The major challenge is that an edge device has limited CPU, RAM, and processing power compared with a desktop computer.

Requirements

Constraint	Requirement
A	Real-time processing, less than 30 FPS latency
B	Robust to varying illumination
C	Low CPU and memory usage
D	Recognize faces from predefined dataset

1.2 Proposed Pipeline

Camera

↓

Frame Acquisition

↓

Resize Frame

↓

Lighting Normalization

↓

Grayscale Conversion

↓

Face Detection

↓

Face Alignment / Cropping

↓

Feature Extraction

↓

Face Recognition

↓

Confidence Check

↓

Display Name / Unknown

↓

Output

Stage 1: Image Acquisition

A camera is connected to the Raspberry Pi.

OpenCV's VideoCapture() can be used to capture frames.

Conceptually:

cap = cv2.VideoCapture(0)

Each frame is processed continuously.

Why?

A live camera allows the system to recognize faces in real time.

Constraint satisfied

Using a camera with an appropriate resolution, such as **640 × 480**, reduces the amount of data that must be processed.

This reduces:

- CPU usage
- Memory usage
- Processing time

Stage 2: Frame Resizing

High-resolution frames require more computation.

Therefore, the input frame can be resized before face detection.

For example:

frame = cv2.resize(frame, (640, 480))

Why?

A smaller frame contains fewer pixels.

This allows the detector to operate faster.

Trade-off

Very small images can reduce recognition accuracy.

Therefore, **640 × 480** or another moderate resolution provides a reasonable balance between speed and accuracy.

Stage 3: Preprocessing

The captured image may contain:

- Shadows
- Bright regions
- Low illumination
- Uneven illumination
- Camera noise

The image can be converted into grayscale:

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

For lighting normalization, histogram equalization can be applied:

gray = cv2.equalizeHist(gray)

Purpose

Grayscale reduces a three-channel color image to one channel.

This reduces computational requirements.

Histogram equalization improves contrast, especially when the image is dark or has poor illumination.

Stage 4: Face Detection

A lightweight detector should be selected for an edge device.

A Haar Cascade classifier can be used:

face_cascade.detectMultiScale()

The detector identifies the bounding box of each face.

Output:

Face detected

x = ...

y = ...

width = ...

height = ...

Why Haar Cascade?

It is relatively lightweight and can work efficiently on CPU-based systems.

A more advanced detector can improve robustness, but it may require additional computational resources.

Stage 5: Face Cropping

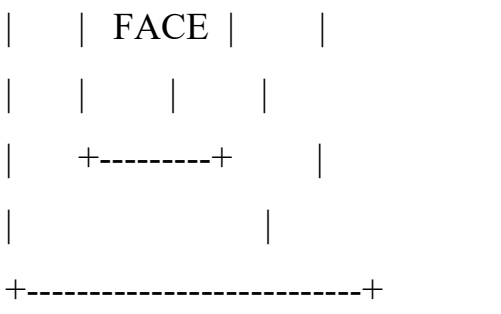
Once a face is detected, only the face region needs to be processed.

Full Frame

+--------------------------+

| |

| +---------+ |



The face region is cropped from the original frame.

Why?

There is no need to perform recognition on the entire image.

Processing only the detected face reduces computation.

Stage 6: Face Normalization

The detected face should be resized to a fixed size.

For example:

face = cv2.resize(face, (100, 100))

The face can also be normalized for brightness and contrast.

Why?

Faces in different frames may have different:

- Sizes
- Brightness
- Positions

Normalization provides a more consistent input to the recognition stage.

Stage 7: Feature Extraction

The system must represent the face using useful features.

For a lightweight system, classical feature extraction methods can be considered.

One suitable approach is **LBPH — Local Binary Patterns Histograms**.

LBPH captures local texture information from the face.

Advantages

- Lightweight
- CPU-friendly
- Works reasonably well with small datasets
- Suitable for simple edge-device applications

Stage 8: Face Recognition

The extracted face features are compared with features stored in the predefined dataset.

The system may produce:

Person: Sreelatha

Confidence: High

or:

Unknown Person

A confidence threshold should be used.

Why?

Without a threshold, the system may incorrectly assign an unknown person to the closest known person.

Therefore:

High confidence → Accept recognition

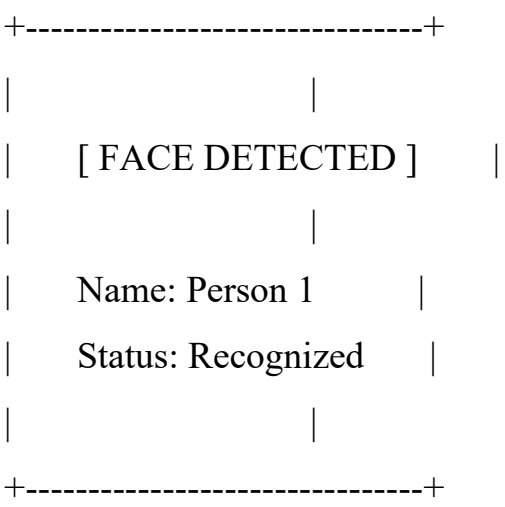
Low confidence → Unknown

This reduces false recognition.

Stage 9: Output

The result is displayed on the live video.

Example:



OpenCV drawing functions can be used:

cv2.rectangle()

cv2.putText()

Constraint Analysis

Constraint A — Real-Time Processing

To maintain real-time performance:

- Reduce frame resolution
- Use lightweight face detection
- Process only detected face regions
- Avoid unnecessary image processing
- Process every second or third frame for detection if necessary
- Use efficient OpenCV functions

Instead of running face detection on every frame:

Frame 1 → Detect

Frame 2 → Track

Frame 3 → Track

Frame 4 → Detect

This can significantly reduce CPU usage.

Constraint B — Varying Lighting

Lighting variations can be handled using:

- Grayscale conversion
- Histogram equalization
- Contrast normalization
- Controlled confidence thresholds

This improves face detection when the scene becomes darker or brighter.

Constraint C — Limited Memory and CPU

The system should:

- Use low-resolution frames
- Avoid storing unnecessary frames
- Use lightweight classifiers
- Process one frame at a time
- Release unused resources

For example:

cap.release()

cv2.destroyAllWindows()

Constraint D — Predefined Dataset

The training dataset contains known people.

Example:

Dataset

```
│
├── Person_1
├── Person_2
├── Person_3
└── Person_4
```

The extracted facial features are stored and compared against incoming faces.

Final Justification

The proposed system satisfies the requirements because it uses **low-resolution acquisition, grayscale preprocessing, lightweight face detection, face-only processing, lightweight feature representation, and threshold-based recognition**.

Therefore, the design is appropriate for a low-power edge device.

2. Automated Video Surveillance with Alert System

2.1 Problem Analysis

The security company wants a system capable of continuously monitoring CCTV video.

The system must:

1. Detect motion.
2. Identify unusual activity.
3. Generate alerts.
4. Work with standard CCTV feeds.
5. Reduce false alarms.

2.2 Complete Pipeline

CCTV Camera

↓

Video Acquisition

↓

Frame Resizing

↓

Noise Reduction

↓

Background Modeling

↓

Motion Detection

↓

Morphological Processing

↓

Object/Feature Detection

↓

Motion Analysis

↓

Unusual Activity Decision

↓

Alert Generation

↓

Display / Recording

Stage 1: CCTV Video Acquisition

OpenCV can receive video from:

- Webcam
- CCTV camera
- Video file
- Network stream

VideoCapture() can be used.

For a video file:

cap = cv2.VideoCapture("video.mp4")

For a camera:

cap = cv2.VideoCapture(0)

Stage 2: Frame Preprocessing

CCTV images may contain:

- Noise
- Compression artifacts
- Low lighting
- Shadows

The frame can be resized and converted to grayscale.

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

A Gaussian filter can reduce noise:

blur = cv2.GaussianBlur(gray, (5,5), 0)

Why?

Noise can create false motion.

Filtering reduces small unwanted variations.

Stage 3: Background Modeling

The system creates a representation of the normal background.

For example:

Normal scene:

Road + Building + Trees + Empty Area

When a new object enters:

Road + Building + Person + Trees

The difference can be detected.

Stage 4: Motion Detection

Background subtraction can be used.

Common OpenCV approaches include:

- MOG2
- KNN background subtraction

Example:

fgmask = backSub.apply(frame)

The foreground mask identifies areas that differ from the background.

Stage 5: Morphological Processing

The foreground mask may contain small noisy regions.

Morphological operations can clean the mask.

Opening

Removes small noise.

Closing

Fills small gaps.

Dilation

Makes detected objects more prominent.

Conceptually:

Raw mask

↓

Noise removal

↓

Morphological operation

↓

Clean foreground

Stage 6: Contour Detection

Contours can be detected using:

cv2.findContours()

The contour represents a detected moving region.

Small contours can be ignored.

For example:

Area < threshold → Ignore

Area > threshold → Possible object

Why?

Leaves, small shadows, and camera noise may otherwise trigger alerts.

Stage 7: Unusual Activity Detection

Motion alone does not necessarily mean suspicious activity.

For example:

Person walking normally → No alert

Person standing in restricted area → Possible alert

Vehicle entering restricted zone → Alert

Large object moving at unusual speed → Alert

Therefore, the system should combine multiple conditions.

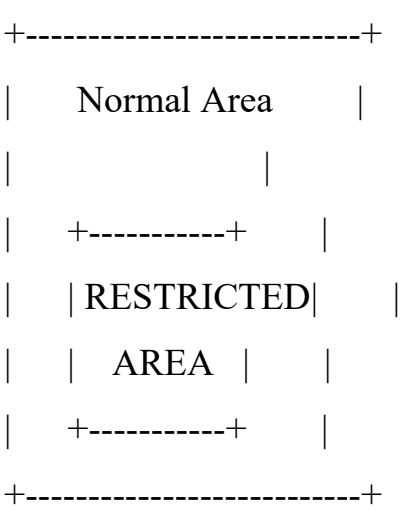
Possible conditions:

- Object size
- Object speed
- Direction
- Region of interest
- Duration
- Time of occurrence

Stage 8: Region of Interest

A restricted region can be defined.

Example:



If movement occurs outside the restricted area, no alert is generated.

If an object enters the restricted region, an alert can be generated.

This significantly reduces false positives.

Stage 9: Alert Generation

If suspicious activity is detected:

ALERT!

Unusual movement detected

Time: 14:20:32

Location: Restricted Zone

The system can:

- Display an alert
- Save the frame
- Record the event
- Send notification through an external service

Handling Dynamic Backgrounds

Dynamic environments contain:

- Moving trees
- Rain
- Shadows
- Changing illumination
- Moving objects

A simple frame-difference method may produce many false positives.

Therefore, adaptive background subtraction such as MOG2 can be used.

Additional filtering and minimum-area thresholds can further reduce false detections.

Real-Time Constraint

To maintain real-time operation:

- Resize frames

- Use efficient background subtraction
- Avoid processing every pixel at high resolution
- Use ROI processing
- Ignore small contours
- Process only important regions

Final Justification

The system minimizes false positives by combining **background subtraction, noise filtering, morphological processing, contour-size filtering, ROI analysis, and temporal motion analysis**.

Therefore, the system is more reliable than simply detecting any movement.

3. OCR System for Low-Quality Documents

3.1 Problem Analysis

OCR stands for **Optical Character Recognition**.

It converts text present in an image into machine-readable text.

The company has low-quality scanned documents containing:

- Noise
- Low resolution
- Uneven illumination
- Different fonts
- Blurred characters
- Background artifacts

Therefore, preprocessing is extremely important before OCR.

3.2 Complete Pipeline

Scanned Document

↓

Image Acquisition

↓

Grayscale Conversion

↓

Noise Reduction

↓

Contrast Enhancement

↓

Thresholding

↓

Morphological Processing

↓

Text Segmentation

↓

Character/Word Region Detection

↓

OCR Engine

↓

Text Post-processing

↓

Final Text Output

Stage 1: Image Acquisition

The scanned document is loaded using OpenCV.

image = cv2.imread("document.jpg")

Stage 2: Grayscale Conversion

The document is converted from RGB/BGR to grayscale.

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

Why?

A color image contains three channels.

Text recognition generally requires intensity information rather than full color.

This reduces computational requirements.

Stage 3: Noise Reduction

Scanned documents may contain:

- Dust
- Salt-and-pepper noise
- Scanner artifacts

A median filter can be useful:

denoised = cv2.medianBlur(gray, 3)

Why?

Noise removal prevents unwanted pixels from being interpreted as characters.

Stage 4: Contrast Enhancement

Low-quality documents may have weak contrast.

Histogram equalization can improve contrast:

enhanced = cv2.equalizeHist(denoised)

This makes the difference between:

Text ↔ Background

more visible.

Stage 5: Thresholding

Thresholding converts the grayscale image into a binary image.

For example:

Original:

White background

Black text

↓

Binary:

0 → Black

255 → White

Adaptive thresholding is particularly useful when illumination varies across the document.

Conceptually:

binary = cv2.adaptiveThreshold(...)

Why adaptive thresholding?

A single global threshold may fail when one side of the document is bright and another side is dark.

Adaptive thresholding determines thresholds based on local regions.

Stage 6: Morphological Processing

Morphological operations can improve character shapes.

Opening

Removes small noise.

Closing

Connects small gaps in characters.

Dilation

Makes thin text thicker.

Erosion

Removes small unwanted regions.

These operations should be applied carefully because excessive processing can distort characters.

Stage 7: Text Segmentation

The system separates text regions from non-text regions.

Possible regions include:

Header

Paragraph

Table

Image

Footer

Contours can be used to identify connected components.

Stage 8: OCR

After preprocessing, the clean text image is passed to an OCR engine.

The OCR engine identifies characters and words.

Example output:

Original image:

NAME: RAVI

AGE: 21

OCR Output:

NAME: RAVI

AGE: 21

Stage 9: Post-processing

OCR results may contain errors.

For example:

Original: COMPUTER

OCR: COMPVTER

Post-processing can use:

- Dictionary checking
- Character correction
- Removal of unnecessary spaces
- Line reconstruction

Batch Processing

The company needs to process many documents.

Instead of keeping all images in memory:

Document 1 → Process → Save

Document 2 → Process → Save

Document 3 → Process → Save

...

Only one or a small number of images should be processed at a time.

This reduces memory consumption.

Handling Varying Fonts

Different fonts have different character shapes.

Good preprocessing helps the OCR engine by producing clear character boundaries.

The system should avoid aggressive erosion or dilation because these operations can change the appearance of characters.

Handling Illumination

Adaptive thresholding is particularly useful.

For example:

Left side: Dark

Right side: Bright

↓

Adaptive Thresholding

↓

Consistent text/background separation

Final Justification

The proposed OCR system handles low-quality documents using **grayscale conversion, denoising, contrast enhancement, adaptive thresholding, morphological processing, text segmentation, and OCR**.

Each stage improves the quality of the image before recognition, thereby increasing OCR accuracy.

For batch processing, processing documents sequentially avoids unnecessary memory consumption.

4. Facial Expression Recognition for Mobile Application

4.1 Problem Analysis

The mobile application needs to recognize:

- Happy
- Sad
- Neutral

The system must operate on a mobile CPU with limited processing power.

The main challenges are:

- Limited CPU
- Real-time requirement
- Slight head movement
- Changing illumination

4.2 Proposed Pipeline

Mobile Camera

↓

Frame Acquisition

↓

Frame Resizing

↓

Face Detection

↓

Face Tracking

↓

Face Cropping

↓

Grayscale / Normalization

↓

Expression Feature Extraction

↓

Classification

↓

Happy / Sad / Neutral

↓

Display Result

Stage 1: Camera Acquisition

The mobile camera continuously captures frames.

However, processing full-resolution frames is expensive.

Therefore, a smaller resolution can be used.

Stage 2: Frame Resizing

For example:

Original:

1920 × 1080

↓

Processing:
640 × 360
This reduces the number of pixels dramatically.

Benefit

- Lower CPU usage
- Lower memory usage
- Faster processing

Stage 3: Face Detection

A lightweight face detector identifies the face.
The detector returns:
x, y, width, height
The bounding box is drawn around the face.

Stage 4: Face Tracking

Instead of detecting the face from scratch on every frame, tracking can be used between detection steps.
Example:
Frame 1 → Detect face
Frame 2 → Track
Frame 3 → Track
Frame 4 → Track

Frame 5 → Detect again

Benefit

This significantly reduces CPU usage.

It also helps handle slight head movements.

Stage 5: Face Cropping

Only the detected face region is extracted.

Full image

↓

Face region

↓

Expression analysis

There is no need to analyze the background.

Stage 6: Preprocessing

The face can be converted to grayscale:

gray_face = cv2.cvtColor(face, cv2.COLOR_BGR2GRAY)

Histogram equalization can help with illumination changes.

The face is resized to a fixed size.

Stage 7: Feature Extraction

Expression-related facial features include:

- Mouth shape
- Eye shape
- Eyebrow position
- Cheek movement

For a lightweight system, facial landmarks or simple image features can be used.

For example:

Happy:

Mouth corners move upward

Sad:

Mouth corners move downward

Neutral:

Little expression movement

Stage 8: Classification

The extracted features are passed to a lightweight classifier.

Output:

Expression: HAPPY

or:

Expression: SAD

or:

Expression: NEUTRAL

Handling Head Movement

Slight head movement can be handled using:

- Face tracking
- Frequent re-detection
- Face alignment
- Landmark-based normalization

The system should not assume that the face remains perfectly stationary.

Handling Lighting Changes

Lighting normalization can include:

- Grayscale conversion
- Histogram equalization
- Contrast normalization

This helps prevent brightness changes from being interpreted as facial-expression changes.

Real-Time Optimization

The system can improve performance using:

1. Lower Resolution

Less data means faster processing.

2. Face-Only Processing

Only the face region is analyzed.

3. Frame Skipping

Expression detection does not necessarily need to run on every camera frame.

4. Lightweight Models

Avoid computationally expensive models when mobile CPU resources are limited.

5. Tracking

Tracking reduces repeated face detection.

Final Justification

The proposed system balances accuracy and efficiency by using **low-resolution input, lightweight face detection, face tracking, face cropping, illumination**

normalization, and lightweight expression classification.

This makes it suitable for a mobile CPU while maintaining real-time performance.

5. Smart Traffic Monitoring System

5.1 Problem Analysis

The city authority needs an automated traffic monitoring system.

The system should:

- Detect vehicles
- Track vehicles
- Count vehicles
- Display traffic statistics
- Operate continuously
- Minimize processing delay

5.2 Complete Pipeline

Traffic Camera

↓

Video Acquisition

↓

Frame Resizing

↓

Preprocessing

↓

Vehicle Detection

↓

Vehicle Tracking

↓

Virtual Counting Line

↓

Vehicle Counting

↓

Statistics Calculation

↓

Visualization

↓

Output

Stage 1: Video Acquisition

The system receives live video from a traffic camera.

OpenCV can capture the stream using:

cap = cv2.VideoCapture(0)

or a network/video source.

The frames are continuously read.

Stage 2: Frame Preprocessing

Traffic video may contain:

- Noise
- Shadows
- Varying illumination
- Camera vibration

The frame can be resized to reduce processing time.

For example:

1920 × 1080

↓

640 × 480

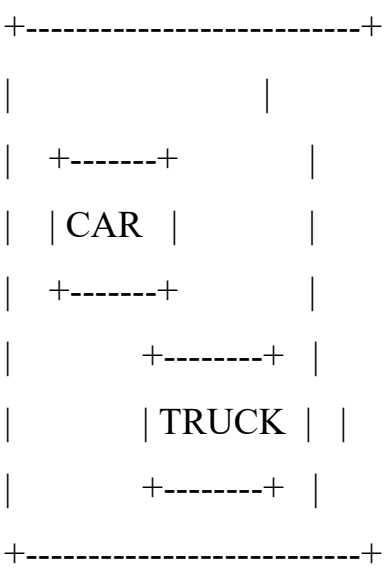
Stage 3: Vehicle Detection

The system must identify:

- Cars
- Buses
- Trucks
- Motorcycles

A vehicle detector identifies the bounding box.

Conceptually:



Stage 4: Vehicle Tracking

Detection tells us:

"There is a vehicle here."

Tracking tells us:

"This is the same vehicle that I detected in the previous frame."

Each vehicle can be assigned an ID.

Example:

Vehicle ID 1 → Car

Vehicle ID 2 → Bus

Vehicle ID 3 → Truck

When the vehicles move across frames, their positions are updated.

Stage 5: Centroid Calculation

The center of each bounding box can be calculated.

For bounding box:

the centroid is:

The centroid is useful for tracking and counting.

Stage 6: Vehicle Counting

A virtual line can be placed across the road.

Example:

---------- ← Counting Line

🚙

↓

🚙

↓

When a vehicle's centroid crosses the line:

Count = Count + 1

This prevents counting the same vehicle repeatedly.

Stage 7: Direction Detection

Vehicle movement direction can also be determined.

For example:

Upward traffic → 25 vehicles

Downward traffic → 31 vehicles

This can be calculated by comparing the current and previous centroid positions.

Stage 8: Statistics

The system can display:

TRAFFIC MONITOR

Cars : 42

Buses : 8

Trucks : 12

Motorcycles: 27

Total : 89

Additional statistics can include:

- Vehicles per minute
- Vehicles per hour
- Direction-wise traffic
- Average speed
- Traffic density

Stage 9: Visualization

OpenCV drawing functions can display information.

Rectangle

cv2.rectangle()

Used to draw vehicle bounding boxes.

Text

cv2.putText()

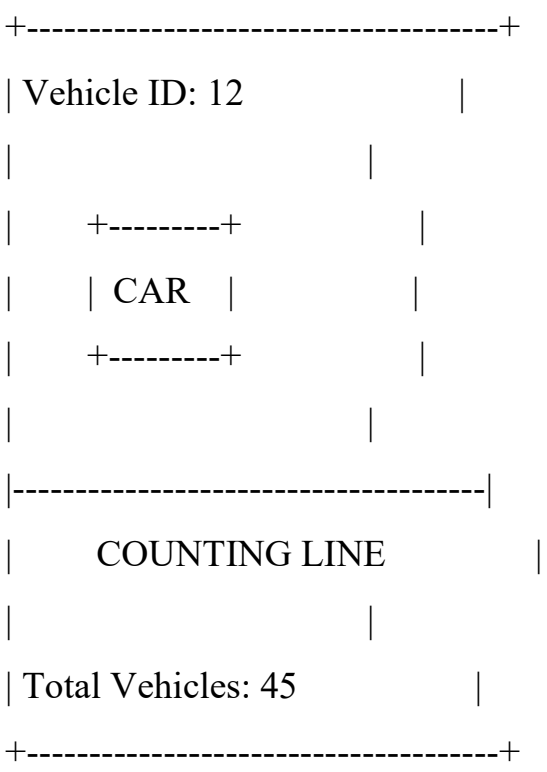
Used to display vehicle IDs and statistics.

Line

cv2.line()

Used to draw the counting line.

Example:



Continuous Operation

The system should run continuously:

Capture Frame

↓

Detect

↓

Track

↓

Count

↓

Display

↓

Next Frame

↓

Capture Frame

↓

...

The loop continues until the operator stops the system.

Real-Time Performance

To minimize delay:

1. Reduce Resolution

Smaller frames require less computation.

2. Use Region of Interest

Only the road region needs to be processed.

For example:

+-------------------------+

| SKY / BUILDING | ← Ignore

|-------------------------|

| |

| ROAD REGION | ← Process

| |

+-------------------------+

3. Track Instead of Detecting Constantly

Detection can be performed periodically while tracking handles intermediate frames.

4. Avoid Unnecessary Processing

Only vehicle regions should be analyzed after detection.

5. Use Efficient Algorithms

Lightweight detection and tracking algorithms are preferred for continuous operation.

Accuracy Considerations

Traffic systems can suffer from:

- Vehicle occlusion
- Shadows
- Night-time conditions
- Rain
- Multiple vehicles overlapping
- Camera vibration

To improve accuracy:

- Use appropriate camera placement
- Use ROI
- Apply filtering

- Use tracking IDs
- Use minimum confidence thresholds
- Avoid counting until a vehicle crosses the defined line

Constraint Analysis

Requirement	Design Solution
Detect vehicles	Vehicle detection
Track vehicles	Object tracking
Count vehicles	Centroid + virtual line
Display statistics	cv2.putText()
Draw bounding boxes	cv2.rectangle()
Continuous operation	Frame-processing loop
Low delay	Resizing + ROI + efficient detection
Reduce duplicate counts	Unique tracking IDs

Final Justification

The proposed traffic monitoring system uses a complete pipeline consisting of **video acquisition, preprocessing, vehicle detection, tracking, centroid calculation, line-crossing detection, counting, and visualization**.

The use of tracking IDs ensures that a vehicle is not counted repeatedly. Frame resizing and ROI processing reduce computational requirements, while OpenCV drawing functions provide real-time visualization.

Therefore, the system satisfies the requirements for **continuous, low-delay traffic monitoring**.

Overall Comparison of the Five Systems

System	Main Input	Detection	Analysis	Final Output	Main Constraint
Face Recognition	Live camera	Face	Face features	Person name	Low CPU
Surveillance	CCTV	Moving objects	Motion/activity	Alert	Low false positives
OCR	Scanned document	Text regions	Character recognition	Extracted text	Low-quality images
Expression Recognition	Mobile camera	Face	Facial features	Happy/Sad/Neutral	Mobile real-time
Traffic Monitoring	CCTV/live video	Vehicles	Tracking/counting	Traffic statistics	Continuous operation

Overall Design Principle

All five systems follow the same basic computer vision methodology:

However, the **specific algorithm at each stage changes according to the constraints**.

For example:

- Face recognition prioritizes **low CPU and memory**.
- Surveillance prioritizes **false-positive reduction**.
- OCR prioritizes **image enhancement and segmentation**.
- Facial expression recognition prioritizes **mobile efficiency and robustness**.

- Traffic monitoring prioritizes **detection, tracking, counting, and continuous real-time operation**.

This demonstrates that in constraint-based computer vision, the best solution is **not necessarily the most complex algorithm**. The best solution is the one that provides an appropriate balance between **accuracy, computational cost, speed, robustness, and resource requirements**.